CalendarApp 仕様書

最終更新: 2026-08-23

このドキュメントはCalendarAppの現在の仕様をまとめたもの。個別の詳細は `docs/` 配下の他ドキュメントに分かれているので、必要に応じて参照すること。

- `docs/DATABASE.md` — テーブル運用・マイグレーションフロー
- `docs/PERMISSIONS.md` — RLSが実際に強制している権限の一覧
- `docs/SECURITY.md` — セキュリティ対策の実施状況と未対応事項
- `docs/DEPLOYMENT.md` — 案件ごとのインフラ払い出し手順
- `docs/TEST_CASES.md` — テストケース一覧

---

1. プロダクト概要

キャバレー・ホステスバー等、水商売系の代理店・店舗向けの多店舗対応業務管理SaaS。店舗ごとに独立したブランド名でデプロイされる（後述のマルチテナント運用を参照）。

主な利用者は2種類:

- **admin（店舗管理者）**: シフト確定、ルーム被りチェック、売上入力、支払通知書発行、キャスト管理、店舗設定
- **cast（キャスト本人）**: シフト提出、修正申請、空き枠応募、自分の売上・報酬確認、支払通知書確認

これに加えてSaaS運営者用の**owner**ロールがあり、`/internal/*` 配下から全店舗の一覧・利用停止/再開・新規店舗申込の承認を行う（`store_id` を持たない、店舗に紐づかない唯一のロール）。

2. 技術スタック

レイヤー	技術
Frontend / Backend	Next.js（App Router）+ TypeScript
UI	Tailwind CSS + shadcn/ui
BaaS	Supabase（Auth / Postgres / Realtime / Storage）
PDF生成	`@react-pdf/renderer`（サーバーサイドでバイト列を直接生成。ブラウザ印刷には依存しない）
メール送信	Resend
Web Push	VAPID鍵ペア + Service Worker（`public/sw.js`）
Google Drive連携	OAuth 2.0（`drive.file` スコープ）。`src/lib/google-drive.ts`
デプロイ	Vercel（Next.jsアプリ）+ Supabase Cloud（DB/Auth）
テスト	Vitest（`tests/rls/security.test.ts` でRLSを実際のPostgRESTに対して検証）

Node.jsの実行環境固有の注意点は `AGENTS.md` を参照（このプロジェクトのNext.jsバージョンは学習データより新しい可能性があるため、挙動に迷ったら `node_modules/next/dist/docs/` の該当ドキュメントを確認すること、という指示が書かれている）。

3. システム構成・マルチテナント運用

このリポジトリは1つだが、**案件（会社）ごとに完全に独立したVercelプロジェクト＋独立したSupabaseプロジェクトを払い出す**運用にしている。理由は `docs/SECURITY.md` を参照（アプリ層のテナント分離が万一破られても、そもそもDBが物理的に別、という多層防御）。

2026-08時点で稼働中のデプロイが2系統ある:

1. **自社運用環境**（GitHubリポジトリ: `rayzyfurukawa/CalendarApp`。git remote名 `origin`）

2. **クライアント向け環境**（GitHubリポジトリ: `eve8008/CalendarApp`。git remote名 `client-eve8008`。専用のVercelプロジェクト・専用のSupabaseプロジェクトを新規に作成して払い出し済み）

コード変更は**両方のリポジトリのmainブランチにpushする**運用になっている（`git push origin main` と `git push client-eve8008 main:main` の両方）。DBマイグレーションも同様に**両方のSupabaseプロジェクトに個別適用が必要**（`supabase link` で対象を切り替えてから `db push`。1つに適用しても他方には自動反映されない）。

具体的なVercelプロジェクト名・Supabase project ref・各種キーは、このリポジトリには含めない方針（`DEPLOYMENT.md` の方針を踏襲）。社内の秘匿情報として別管理すること。

クライアント環境固有の既知の制約

クライアント側Vercelプロジェクトは無料のHobbyプランを使っている。**Hobbyプランは非公開GitHubリポジトリでの複数人コラボレーションをサポートしていない**ため、リポジトリのコミット作者（Vercelプロジェクトのメンバーではないアカウント）がpushしても自動デプロイが `Deployment Blocked` になり、本番に反映されない。当面の回避策は、pushの都度Vercelダッシュボードで手動「Redeploy」を押してもらうこと。恒久対応（Proプランへのアップグレード、または常に手動Redeployする運用の継続）は未確定。

4. ロールと権限

`public.users.role` は `user_role` enum: `'admin' | 'cast' | 'owner'`。

権限の最終的な境界は常にPostgreSQLのRow Level Security（RLS）で決まる（UI上の表示制御はその上澄み）。3層防御の構造・テーブルごとのアクセス範囲の一覧は `docs/PERMISSIONS.md` を参照。要点だけ書くと:

1. サーバー側ガード（Server Component / Route Handlerで `role` を再確認してから処理）
2. RLS（`supabase/migrations/` 内の `create policy` が正）
3. 列単位トリガー（`users.role` / `store_id` / `reward_rate` / `is_active` は本人が自分の行を更新する場合のみDBトリガーで変更前の値に戻される。RLSは行単位の制御であり列単位ではないため、これがないと一般キャストが自分の `role` を `admin` に書き換えられてしまう）

`owner` ロールは `(app)` 配下の通常シェルを経由せず、`(app)/layout.tsx` で直接 `/internal/stores` にリダイレクトされる。これらの画面は `isPlatformOwner()`（`src/lib/platform-owner.ts`）で判定した独自レイアウトのため、通常のログアウトボタン（`AppNav` 内）を持たない。専用の `InternalLogoutButton`（`src/components/internal/logout-button.tsx`）で対応している。

5. データベース設計

Supabase（Postgres）。**スキーマの正はこの文書ではなく `supabase/migrations/*.sql` そのもの**。マイグレーションはタイムスタンプ順に適用され、原則として一度適用したファイルは編集しない（変更が必要なら新しいファイルを追加する）。運用フローの詳細は `docs/DATABASE.md`。

5.1 ER関係の概要

```
stores 1──┬─1 store_settings
          ├─1 store_google_drive_tokens
          ├─* areas ─* rooms
          ├─* users (role: admin / cast。ownerはstore_idなし)
          ├─* shifts ─* shift_modification_requests
          ├─* vacant_slots ─* vacant_slot_applications
          ├─* sales
          ├─* reward_calculations (未使用。5.3参照)
          ├─* shift_checklist_items
          ├─* cast_profile_fields
          └─* notifications

users(cast) 1─* shifts, sales, vacant_slot_applications,
              notifications, push_subscriptions
```

`auth.users`（Supabase Auth管理）と `public.users` は1:1（`id` が同じ）。`handle_new_user()` トリガーが `auth.users` へのINSERT時に `public.users` へ最小限の行（id, email）を自動作成し、その後アプリ側（admin API）で `role` / `store_id` / `display_name` 等を更新する2段階構成。

5.2 全テーブル定義

以降、`?` は `NULL` 許容、`PK` は主キー、`FK→table` は外部キー。デフォルト値は主なもののみ記載。

`stores` — 店舗（テナント）本体

カラム	型	説明
`id`	uuid PK	
`name`	text	店舗名
`is_active`	boolean（既定true）	falseで利用停止（owner操作）
`created_at`	timestamptz	

`store_settings` — 店舗ごとの設定（`store_id` にUNIQUE、実質1:1）

カラム	型	説明
`id`	uuid PK	
`store_id`	uuid FK→stores	
`deadline_type`	enum `deadline_type`（`monthly` / `weekly` / `none`）	シフト締切ルールの種別
`deadline_day`	int	月次=日付(1-31) / 週次=曜日
`deadline_week_start_day`	int	週次締切の起算曜日
`require_approval_after_deadline`	boolean	締切後修正に承認が必要か
`location_mode`	text	`area` / `room` / `both` / `none` など、場所管理の単位切り替え
`area_label` / `room_label`	text	画面上の表示ラベルのカスタム文言
`theme`	text	管理画面の配色テーマ（`default` / `rose` / `midnight` / `emerald` / `gold` / `violet`）
`logo_url`	text?	支払通知書等に表示するロゴ
`usd_jpy_rate`	numeric?	海外プラットフォーム売上の円換算に使うUSD/JPYレート
`platform_settings`	jsonb（既定 `{}`）	プラットフォーム別（stripchat/fc2/dxlive）の換算方式設定。`src/lib/platform-earnings.ts` 参照
`created_at` / `updated_at`	timestamptz	

注意: `store_settings` のSELECTポリシーは店舗所属の全ユーザー（castも含む）に開いている。秘匿情報（トークン等）をこのテーブルに追加してはいけない（→ `store_google_drive_tokens` が別テーブルになっている理由）。

`users` — 認証ユーザーのプロフィール（`auth.users` と1:1）

カラム	型	説明
`id`	uuid PK（`auth.users.id` と同一）	
`email`	text	
`login_id`	text	ログインID（メールの代わりにIDでログインできる仕組み。`/api/auth/resolve-login-id` でメールに解決）
`role`	enum `user_role`（`admin` / `cast` / `owner`）	
`store_id`	uuid? FK→stores	ownerのみnull
`display_name`	text?	源氏名
`real_name`	text?	本名（支払通知書に使用）
`photo_url`	text?	
`phone`	text?	
`notes`	text?	
`reward_rate`	numeric(5,4)（既定0.4）	基本報酬率
`is_active`	boolean（既定true）	
`birthdate`	date?	
`show_birthdate`	boolean	プロフィール上での公開可否
`bank_name` / `bank_branch` / `bank_account_number`（7桁数字制約） / `bank_account_holder`	text?	振込先。支払通知書に使用
`monthly_goal`	numeric?	キャスト自身の月間目標
`period_start_date` / `period_end_date`	date?	稼働期間（プロフィール項目）
`profile_answers`	jsonb（既定 `{}`）	`cast_profile_fields` で定義したカスタム項目の回答
`accent_font` / `accent_skin`	text	キャスト向けUIの着せ替え設定
`calendar_token`	text?	外部カレンダー連携用トークン（`/api/calendar/[token]` のics配信で使用）
`created_at` / `updated_at`	timestamptz	

`role` / `store_id` / `reward_rate` / `is_active` は権限に関わるため、本人自身による更新時はDBトリガーで元の値に強制される（`SECURITY.md` 参照）。

`areas` / `rooms` — 場所の単位

`store_settings.location_mode` によりどちらを使うか（両方・片方・なし）が変わる。

- `areas`：`id`, `store_id` FK, `name`, `sort_order`, `is_active`, `created_at`
- `rooms`：`id`, `store_id` FK, `area_id`？FK→areas, `name`, `capacity`？（正の数チェック制約）, `notes`？, `is_active`, `created_at`

`shifts` — シフト本体

カラム	型	説明
`id`	uuid PK	
`store_id`	uuid FK→stores	
`cast_id`	uuid FK→users	
`date`	date	
`start_time` / `end_time`	time	`end_time <= start_time` は日跨ぎシフトとして正規化して扱う
`is_period_day`	boolean	生理日等の申告フラグ
`room_id`	uuid? FK→rooms	
`area_id`	uuid? FK→areas	
`status`	enum `shift_status`（`draft` / `submitted` / `confirmed` / `rejected` / `modification_requested`）	
`checklist_answers`	jsonb（既定 `{}`）	`shift_checklist_items` への回答
`notes`	text?	キャストの備考（管理者・キャスト共有）
`admin_note`	text?	管理者専用メモ
`confirmed_at` / `confirmed_by`	timestamptz? / uuid? FK→users	
`created_at` / `updated_at`	timestamptz	

重要な制約: `shifts_no_double_booking`（GiST EXCLUDE制約、`btree_gist` 拡張使用）。同一 `store_id` + `room_id` で時間帯が重なる `status='confirmed'` シフトはINSERT/UPDATE時点でDBが拒否する（エラーコード `23P01`）。日跨ぎシフトも正しく判定される。以前はUI上の事後警告のみだったため、同時操作や直接APIコールでの被りを防げていなかった不具合をDBレベルで修正済み。

`shift_modification_requests` — 確定後のシフト変更申請

`id`, `shift_id` FK→shifts, `requested_by` FK→users, `new_date`?, `new_start_time`?, `new_end_time`?, `new_room_id`? FK→rooms, `reason`, `status`（enum `request_status`: `pending` / `approved` / `rejected`）, `reject_reason`?, `reviewed_by`? FK→users, `reviewed_at`?, `created_at`, `updated_at`

`vacant_slots` / `vacant_slot_applications` — 空き枠と応募

- `vacant_slots`: `id`, `store_id` FK, `room_id`? FK→rooms, `date`, `start_time`, `end_time`, `capacity`（既定1、正の数チェック制約）, `is_published`, `notes`?, `created_by` FK→users, `created_at`
- `vacant_slot_applications`: `id`, `vacant_slot_id` FK→vacant_slots, `cast_id` FK→users, `status`（`request_status`）, `shift_id`? FK→shifts（承認後に生成されたシフトへのリンク）, `reviewed_by`?, `reviewed_at`?, `created_at`

`sales` — 売上（プラットフォーム別・手入力）

カラム	型	説明
`id`	uuid PK	
`store_id`	uuid FK→stores	
`cast_id`	uuid FK→users	
`shift_id`	uuid? FK→shifts	
`date`	date	
`amount`	numeric(12,0)（既定0）	円換算後の売上金額。手入力ならそのまま、プラットフォーム経由なら `raw_units × 換算レート` の計算結果を保存（都度再計算ではなく確定値として保存する設計）
`platform`	text（既定 `manual`）	`manual` / `stripchat` / `fc2` / `dxlive`
`raw_units`	numeric(12,2)?	プラットフォーム上の生の単位数（TK/pt等）。手入力時はnull
`usd_amount`	numeric(12,2)?	換算時のUSD建て金額（参考値）
`fx_rate_snapshot`	numeric(10,4)?	換算時に使用したUSD/JPYレートのスナップショット。事後の一括レート変更機能（`SalesInput` の「為替を一括変更」）で更新される
`is_daily_payment`	boolean（既定 false）	日払い済みフラグ
`rate_override`	numeric(12,4)?	日払い時の単価上書き（1単位＝◯円）
`reward_rate_override`	numeric(5,4)?	日払い時の報酬率上書き。ブラウザのlocalStorageにテンプレとして保存され次回選択できる（`SalesInput` 内 `rateTemplates`）
`daily_payment_at`	timestamptz?	
`worked_hours`	numeric?	稼働時間（登録された行のみ報酬ダッシュボードの時給計算対象になる）
`bonus` / `deduction`	numeric(12,0)（既定0）	
`notes`	text?	
`created_by`	uuid? FK→users	
`created_at` / `updated_at`	timestamptz	

換算式（`src/lib/platform-earnings.ts`）: STRIPCHAT = TK ÷ 10 ÷ 2（USD）、DXライブ = ポイント × 2ドル × 0.3、FC2 = ポイントがそのまま円（USDレート不使用）。店舗ごとに `store_settings.platform_settings` で計算式/固定単価モードを上書き可能。

`reward_calculations` — 未使用テーブル

月次報酬計算結果を保存する想定で作られたが、**現行実装からは一切参照されていない**。報酬額・支払通知書は `sales` から都度ライブ計算している（`src/lib/payment-notice-data.ts` の `fetchNoticeContent`）。将来これを実際の集計キャッシュとして使うか、テーブルごと削除するかは要判断。RLSポリシーだけは他テーブルと同様に張られている。

`notifications` — アプリ内通知

`id`, `store_id`? FK→stores, `user_id` FK→users（通知の宛先）, `title`, `body`?, `link`?（関連画面へのパス。支払通知書の場合は `?cast_id=&from=&to=` をクエリに含む）, `read_at`?（未読管理）, `created_at`

「発行済み支払通知書」一覧（`/admin/issued-notices`）は専用テーブルを持たず、`title = '支払通知書が届きました'` の `notifications` レコードを実体として扱い、`link` のクエリパラメータから `cast_id` / `from` / `to` を都度パースしている。

`shift_checklist_items` — シフト提出時の確認項目（店舗ごとにカスタム）

`id`, `store_id` FK, `label`, `item_type`（既定 `checkbox`）, `options`（text[]、既定空配列）, `sort_order`, `is_active`, `created_at`

`push_subscriptions` — Web Push購読情報

`id`, `user_id` FK→users, `endpoint`, `p256dh`, `auth`, `created_at`

`cast_profile_fields` — キャストプロフィールのカスタム項目定義

`id`, `store_id` FK, `label`, `sort_order`, `is_active`, `created_at`

`store_signup_requests` — セルフサーブ新規店舗申込

`id`, `store_name`, `contact_name`, `contact_email`, `phone` ?, `message` ?, `status`（text、既定 `pending`）, `created_store_id` ? FK→stores（承認後に作成された店舗）, `reviewed_by` ? FK→users, `reviewed_at` ?, `reject_reason` ?, `created_at`

RLSポリシーなし＝`service_role`専用（`/api/store-signup`と`/api/internal/store-signups/[id]/approve`がadmin clientで直接操作する）。`/apply`ページから申込 → `owner`が`/internal/signups`で承認 → `stores`行作成・`store_settings`作成・招待メール送信・admin用`users`行更新、という流れ。

`store_google_drive_tokens` — Google Drive連携用トークン（新規・2026-08-23追加）

カラム	型	説明
`store_id`	uuid PK/FK→stores（1:1）	
`refresh_token`	text	Google OAuthのリフレッシュトークン
`root_folder_id`	text?	アップロード先ルートフォルダのDrive上のID（初回アップロード時にキャッシュ）
`connected_by`	uuid? FK→users	連携操作をしたadmin
`created_at` / `updated_at`	timestamptz	

`store_settings`とは意図的に別テーブルにしている（`store_settings`のSELECTポリシーが店舗所属の全ユーザーに開いているため、そこにトークンを置くとcastからも読めてしまう）。RLSはadmin限定の`for all`のみ。詳細は6.3節。

5.3 RLSの設計方針（要約。詳細は`PERMISSIONS.md`）

- `my_role()` / `my_store_id()`：`public.users`から自分のrole/store_idを引くSECURITY DEFINER関数。ポリシー内で多用される
- 基本パターン: adminは`store_id = my_store_id() and my_role() = 'admin'`で自店舗を全操作、castは`cast_id = auth.uid()`や`user_id = auth.uid()`で自分の行のみ
- `for all`ポリシーは明示的な`WITH CHECK`を書かない限りUSING句がそのまま適用される（更新後の行にも同じ条件が課される）
- 秘匿情報を含むテーブルは、既存の緩いSELECTポリシーを持つテーブルに相乗りさせない（`store_google_drive_tokens`の設計判断）
- 列単位の権限制御はRLSでは不可能なのでトリガーで別途対応する（`users`の`lock_privileged_user_columns()`が実例）

5.4 マイグレーション運用フロー（詳細は`DATABASE.md`）

1. 適用済みファイルは編集しない。新しいタイムスタンプ付きファイルを追加する
2. `supabase migration up --local`でローカル検証
3. `supabase gen types typescript --local > src/lib/supabase/database.types.ts`で型を再生成（**CLIの標準出力にバージョン通知等の余計な行が混入することがあるため、生成後は必ずファイル先頭・末尾を確認して不要な行を削除すること**。今回の実装時に実際に混入し、`tsc`がパースエラーを起こした）
4. `npx tsc --noEmit` / `npm run lint` / `npm run build`を通す
5. 案件ごとに`supabase link --project-ref <ref>`で対象を切り替えてから`supabase db push --linked`（または`--password`指定）で本番に適用。**複数案件を運用している場合、1つに適用しても他方には自動反映されない**ので、両方に適用する

6. 主要機能

6.1 シフト管理

- キャストが日付・時間・ルーム/エリア・チェックリスト回答を入力して提出（`draft` → `submitted`）
- 管理者がルーム被りチェック（自動検知、確定時はDB制約でも強制）をした上で確定/差し戻し
- 確定後の変更は修正申請フロー経由（`shift_modification_requests`）。管理者は「承認してキャンセル」等、承認と同時にステータス変更する操作にも対応
- リアルタイム更新（Supabase Realtime）でシフト状態変化を画面に即時反映
- 管理者向けカレンダー連携（ics配信、`calendar_token`ベースの認証不要URL）

6.2 売上・報酬・支払通知書

- 売上は手入力またはプラットフォーム別（STRIPCHAT/FC2/DXライブ）入力。プラットフォーム分はUSD/JPYレートで円換算して保存
- 日払い機能: 個別の報酬率上書き・単価上書き。テンプレ機能で過去に使った報酬率上書き値をワンタップ選択可能。チェックボックスで複数選択し一括で日払い済みに変更する機能あり
- 為替レート一括変更: 既に登録済みの売上について、絞り込み条件に一致するプラットフォーム売上の金額を新しいレートで再計算・更新する機能（`sales-input.tsx`の「為替を一括変更」）
- 支払通知書PDF: `src/lib/payment-notice-pdf.tsx`（react-pdf）でサーバー生成。ファイル名は`{開始日}-{終了日}_{支払額}_{本名}_支払通知書.pdf`形式に統一（`noticeFileName()` in `payment-notice-data.ts`）
- 発行済み支払通知書の一括ダウンロード（ZIP）、および個別のPDFダウンロード（ブラウザ印刷は使わない。共有時にレイアウトが崩れる問題があったため撤去済み）
- **Google Drive連携**: 発行済み支払通知書を選択してGoogle Driveへ直接アップロード保存できる。年月フォルダに自動整理される（`src/lib/google-drive.ts`、`src/app/api/google-drive/*`）

6.3 通知・PWA

- アプリ内通知（`notifications`テーブル）、既読管理
- Web Push通知（VAPID）。iOS Safari PWAでのアプリアイコンバッジ表示に対応（`navigator.setAppBadge()`は引数なし呼び出しだと反映されないことがあるため、必ず実際の未読件数を明示的に渡す実装にしている。`public/sw.js`参照）
- PWAインストール導線・通知許可導線をキャストのシフト画面に表示

6.4 マルチテナント管理（owner）

- `/internal/stores`: 全店舗の一覧・利用停止/再開
- `/internal/signups`：`/apply`
  から届いた新規店舗申込のレビュー・承認（承認すると店舗・管理者アカウント作成、招待メール送信）